

```

1: //Insertion and Deletion in Doubly Circular Linked List
2:
3: #include <stdio.h>
4: #include <stdlib.h>
5:
6: struct Node
7: {
8:     int data;
9:     struct Node* next;
10:    struct Node* prev;
11: };
12:
13: struct Node* head = NULL;
14:
15: void insertAtBeginning(int data)
16: {
17:     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
18:     newNode->data = data;
19:     newNode->next = head;
20:     newNode->prev = NULL;
21:
22:     if (head == NULL)
23:     {
24:         head = newNode;
25:         newNode->next = head;
26:         newNode->prev = head;
27:     }
28:     else
29:     {
30:         struct Node* tail = head->prev;
31:         head->prev = newNode;
32:         tail->next = newNode;
33:         newNode->prev = tail;
34:         head = newNode;
35:     }
36:     printf("%d inserted at the beginning\n", data);
37: }
38:
39: void insertAtEnd(int data)
40: {
41:     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
42:     newNode->data = data;
43:     newNode->next = head;
44:     newNode->prev = NULL;
45:
46:     if (head == NULL)
47:     {
48:         head = newNode;
49:         newNode->next = head;

```

```

50:         newNode->prev = head;
51:     }
52:     else
53:     {
54:         struct Node* tail = head->prev;
55:         tail->next = newNode;
56:         newNode->prev = tail;
57:         head->prev = newNode;
58:         newNode->next = head;
59:     }
60:     printf("%d inserted at the end\n", data);
61: }
62:
63: void insertAtPosition(int data, int position)
64: {
65:     if (position < 1)
66:     {
67:         printf("Invalid position\n");
68:         return;
69:     }
70:     if (head == NULL && position > 1)
71:     {
72:         printf("List is empty, cannot insert at position %d\n", position);
73:         return;
74:     }
75:
76:     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
77:     newNode->data = data;
78:
79:     if (position == 1)
80:     {
81:         newNode->next = head;
82:         newNode->prev = NULL;
83:         if (head == NULL)
84:         {
85:             head = newNode;
86:             newNode->next = head;
87:             newNode->prev = head;
88:         }
89:         else
90:         {
91:             struct Node* tail = head->prev;
92:             head->prev = newNode;
93:             tail->next = newNode;
94:             newNode->prev = tail;
95:             head = newNode;
96:         }
97:         printf("%d inserted at position %d\n", data, position);
98:         return;

```

```

99:     }
100:
101:     struct Node* current = head;
102:     int currentPosition = 1;
103:     while (currentPosition < position - 1 && current != NULL)
104:     {
105:         current = current->next;
106:         currentPosition++;
107:     }
108:
109:     if (current == NULL || current->next == head)
110:     {
111:         printf("Invalid position\n");
112:         return;
113:     }
114:
115:     newNode->next = current->next;
116:     newNode->prev = current;
117:     current->next->prev = newNode;
118:     current->next = newNode;
119:     printf("%d inserted at position %d\n", data, position);
120: }
121:
122:
123: void deleteAtBeginning()
124: {
125:     if (head == NULL)
126:     {
127:         printf("List is empty!\n");
128:         return;
129:     }
130:     struct Node* temp = head;
131:     if (head->next == head)
132:     {
133:         head = NULL;
134:     }
135:     else
136:     {
137:         head = head->next;
138:         head->prev = temp->prev;
139:         head->prev->next = head;
140:     }
141:     free(temp);
142:     printf("Node deleted from the beginning!\n");
143: }
144:
145: void deleteAtEnd()
146: {
147:     if (head == NULL)

```

```

148:     {
149:         printf("List is empty!\n");
150:         return;
151:     }
152:     struct Node* temp = head;
153:     if (head->next == head)
154:     {
155:         head = NULL;
156:     }
157:     else
158:     {
159:         while (temp->next != head)
160:         {
161:             temp = temp->next;
162:         }
163:         temp->prev->next = head;
164:         head->prev = temp->prev;
165:     }
166:     free(temp);
167:     printf("Node deleted from the end!\n");
168: }
169:
170: void deleteAtPosition(int position)
171: {
172:     if (head == NULL)
173:     {
174:         printf("List is empty!\n");
175:         return;
176:     }
177:     if (position == 1)
178:     {
179:         deleteAtBeginning();
180:         return;
181:     }
182:     struct Node* current = head;
183:     for (int i = 1; i < position; i++)
184:     {
185:         current = current->next;
186:         if (current == head)
187:         {
188:             printf("Invalid position!\n");
189:             return;
190:         }
191:     }
192:     current->prev->next = current->next;
193:     current->next->prev = current->prev;
194:     free(current);
195:     printf("Node deleted from position %d!\n", position);
196: }

```

```
197:
198: void display()
199: {
200:     if (head == NULL)
201:     {
202:         printf("List is empty!\n");
203:         return;
204:     }
205:     struct Node* current = head;
206:     printf("Nodes of the circular doubly linked list: ");
207:     do
208:     {
209:         printf("%d ", current->data);
210:         current = current->next;
211:     } while (current != head);
212:     printf("\n");
213: }
214: int main()
215: {
216:     insertAtBeginning(3);
217:     insertAtEnd(5);
218:     insertAtBeginning(7);
219:     insertAtPosition(9,2);
220:
221:     display();
222:
223:     deleteAtPosition(2);
224:
225:     display();
226:
227:     deleteAtPosition(4);
228:
229:     display();
230:
231:     return 0;
232: }
```